Postman Guide: Async API Testing

This guide explains how to handle **asynchronous API testing** in Postman, including polling patterns, callbacks/webhooks, and scenarios where API responses are delayed or change over time.

---

1. Key Concepts

Asynchronous APIs

- Async APIs do **not complete immediately**.
- Examples: Order processing, background jobs, file uploads, long-running tasks.
- You may receive:
    1. **Immediate acknowledgment** (202 Accepted) → task is processing
    2. **Delayed final result** → success/failure later

Polling Pattern

- Polling is a technique where you repeatedly call a **status endpoint** to check if the task is complete.

Steps:

1. Send initial request to start async process → receive taskId
2. Poll GET /status/:taskId at intervals
3. Stop polling when:
    - Task completes successfully

- Task fails
- Timeout reached

Example Postman Test Script (Polling Simulation):

```
// Get taskId from previous response
let taskId = pm.environment.get("taskId");

// Poll the status API
pm.sendRequest(`https://api.example.com/status/${taskId}`, function (err, res) {
  if (err) {
    console.log("Error polling:", err);
    return;
  }

  let status = res.json().status;
  console.log("Current status:", status);

  if (status === "completed") {
    console.log("Task completed successfully!");
    pm.environment.set("taskStatus", "completed");
  } else if (status === "failed") {
    console.log("Task failed!");
    pm.environment.set("taskStatus", "failed");
  } else {
    console.log("Task still in progress, retry later...");
  }
});
```

> ⚠️ Note: Postman does not natively “wait” in scripts. Polling is **best done manually or via Newman loops** or **external scripting**.

---

2. Callbacks / Webhooks

- Webhooks are used to **notify your system when the task completes**.
- Instead of polling, the server sends a POST request to your **callback URL**.

Conceptual Flow:

1. Client triggers async task
2. Server responds with 202 Accepted + taskId
3. Server processes task asynchronously
4. Server calls your webhook URL with result once ready

Testing Approach in Postman:

- Postman cannot directly act as a live webhook listener
- Options:
    1. Use a **mock server** in Postman to simulate webhook calls
    2. Test payload format and validation after receiving simulated webhook
    3. Combine with polling if webhook not available

---

3. Fake Async Testing Scenarios

Scenario 1: Status API completes later

- Example: POST /startTask → returns taskId
- GET /status/:taskId initially returns "pending" → later "completed"

Testing Approach:

1. Trigger task → save taskId in environment
2. Use **manual or automated polling**

3. Assert status is "completed" or "failed"

Scenario 2: API success now, failure 5 minutes later

- Some tasks may **succeed immediately** but fail later due to delayed checks.

Testing Approach:

1. Trigger task → receive immediate response (success)
2. Poll status endpoint periodically for 5–10 minutes
3. Validate:
    - Final status reflects reality
    - Errors are handled gracefully
    - System can retry or rollback if needed

Postman Tip:

- Save taskId → use **Collection Runner + delay** to simulate waiting
- Newman CLI can be scripted to **run multiple iterations** with wait intervals

---

4. Best Practices

- Use **environment variables** to store taskId and statuses
- Define **polling intervals and maximum retries** to avoid infinite loops
- Always check both **immediate response** and **final status**

- Log all intermediate states for debugging
- Combine polling and webhook testing where possible